

OSSP-SKILL-WEEK12

SESSION23:

Task1: Develop a program that launches multiple jobs connected through a pipeline, monitors their execution, detects failures, and measures execution performance.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define JOBS 3

int main()
{
    int pipefd[JOBS - 1][2];
    pid_t pids[JOBS];

    struct timespec start, end;

    printf("Skill Week 12 - Session 23 - Task 1\n");
    printf("-----\n");
    printf("Executing a large pipeline with %d jobs.\n\n", JOBS);

    clock_gettime(CLOCK_MONOTONIC, &start);

    /* Create pipes */
    for (int i = 0; i < JOBS - 1; i++)
    {
        if (pipe(pipefd[i]) == -1)
        {
            perror("pipe");
            return 1;
        }
    }

    /* Create processes */
    for (int i = 0; i < JOBS; i++)
    {
        pids[i] = fork();

        if (pids[i] < 0)
        {
            perror("fork");
            return 1;
        }

        if (pids[i] == 0)
        {
            /* First job */
            if (i == 0)
```

GNU nano 7.2

```
    if (i == 0)
    {
        dup2(pipefd[0][1], STDOUT_FILENO);
    }

    /* Middle job */
    else if (i < JOBS - 1)
    {
        dup2(pipefd[i - 1][0], STDIN_FILENO);
        dup2(pipefd[i][1], STDOUT_FILENO);
    }

    /* Last job */
    else
    {
        dup2(pipefd[i - 1][0], STDIN_FILENO);
    }

    /* Close all pipe descriptors */
    for (int j = 0; j < JOBS - 1; j++)
    {
        close(pipefd[j][0]);
        close(pipefd[j][1]);
    }

    if (i == 0)
    {
        execlp("seq", "seq", "1", "1000", NULL);
    }
    else if (i == 1)
    {
        execlp("grep", "grep", "[0-9]*$", NULL);
    }
    else
    {
        execlp("wc", "wc", "-l", NULL);
    }

    perror("exec failed");
    exit(1);
}

/* Parent closes pipes */
for (int i = 0; i < JOBS - 1; i++)
{
```

```

for (int i = 0; i < JOBS - 1; i++)
{
    close(pipefd[i][0]);
    close(pipefd[i][1]);
}

/* Wait for all jobs */
int failures = 0;

for (int i = 0; i < JOBS; i++)
{
    int status;

    waitpid(pids[i], &status, 0);

    if (WIFEXITED(status) &&
        WEXITSTATUS(status) == 0)
    {
        printf("Job %d completed successfully.\n", i + 1);
    }
    else
    {
        printf("Job %d failed.\n", i + 1);
        failures++;
    }
}

clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1000000000.0;

printf("\nPipeline execution completed.\n");
printf("Total execution time: %.6f seconds\n", elapsed);
printf("Successful jobs: %d\n", JOBS - failures);
printf("Failed jobs: %d\n", failures);

if (failures == 0)
    printf("Pipeline status: STABLE\n");
else
    printf("Pipeline status: FAILURE DETECTED\n");

return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session23_task1.c
root@LAPTOP-1COJKKG6:~# gcc session23_task1.c -o session23_task1
root@LAPTOP-1COJKKG6:~# ./session23_task1
Skill Week 12 - Session 23 - Task 1
-----
Executing a large pipeline with 3 jobs.

Job 1 completed successfully.
Job 2 completed successfully.
1000
Job 3 completed successfully.

Pipeline execution completed.
Total execution time: 0.026033 seconds
Successful jobs: 3
Failed jobs: 0
Pipeline status: STABLE
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how multiple processes can be connected using pipes to create large pipelines. I understood how fork(), pipe(), dup2(), exec(), and waitpid()

work together for process execution, monitoring, failure detection, and performance measurement.

Task2: Document the shell architecture, create a flow diagram, explain execution paths, describe system calls, record design decisions, and review the documentation.

Source code:

```
GNU nano 7.2
# Skill Week 12 - Session 23 - Task 2

## Project Architecture

The shell system is divided into several components:

1. Input Module
2. Parser Module
3. Job Management Module
4. Process Creation Module
5. Pipeline Module
6. Signal Handling Module
7. Execution Module

## Architecture Diagram

User
|
v
Input Module
|
v
Parser
|
v
Command / Arguments
|
v
Job Management
|
v
Process Creation
|
+-----+
|         |
v         v
Foreground Job   Background Job
|               |
v               v
Execution       Job Table
|
v
Pipeline
|
v
Output
```

```
GNU nano 7.2
## Execution Path

The user enters a command.

The parser identifies the command and its arguments.

The shell creates a process using fork().

The child process executes the required command using exec().

For pipelines, pipe() creates communication channels between processes.
dup2() redirects standard input and output to the appropriate pipe.

The parent process uses waitpid() to monitor child processes.

## Important System Calls

### fork()

Creates a new child process.

### exec()

Replaces the child process with the requested program.

### pipe()

Creates a communication channel between processes.

### dup2()

Redirects standard input or output.

### waitpid()

Allows the parent to wait for and monitor a child process.

### kill()

Sends signals to processes when required for job control.

## Pipeline Execution

Example:
|
```

```

GNU nano 7.2
the second command.

The output of the second command becomes the input of
the third command.

## Design Decisions

- Pipes are used for inter-process communication.
- fork() is used to create independent processes.
- exec() is used to execute external commands.
- waitpid() is used to monitor process completion.
- Separate process handling makes pipeline execution possible.
- Error conditions are checked after system calls.
- Execution time is measured to analyze performance.

## Error Handling

The program checks for failures from:

- pipe()
- fork()
- dup2()
- exec()
- waitpid()

If a system call fails, an error message is displayed.

## Performance Analysis

The total execution time is measured using
clock_gettime().

Performance can be compared by:

- Number of pipeline stages
- Amount of input data
- Number of simultaneous jobs
- Process creation overhead
- Pipe communication overhead

## Conclusion

The system demonstrates how a shell creates processes,
connects them using pipes, executes commands, monitors
their completion, and handles failures.

```

Output:

```
root@LAPTOP-1COJKKG6:~# nano session23_task1.c
root@LAPTOP-1COJKKG6:~# gcc session23_task1.c -o session23_task1
root@LAPTOP-1COJKKG6:~# ./session23_task1
Skill Week 12 - Session 23 - Task 1
```

Executing a large pipeline with 3 jobs.

```
Job 1 completed successfully.
Job 2 completed successfully.
1000
Job 3 completed successfully.
```

```
Pipeline execution completed.
Total execution time: 0.026033 seconds
Successful jobs: 3
Failed jobs: 0
Pipeline status: STABLE
root@LAPTOP-1COJKKG6:~# nano session23_task2.md
root@LAPTOP-1COJKKG6:~# cat session23_task2.md
# Skill Week 12 - Session 23 - Task 2
```

Project Architecture

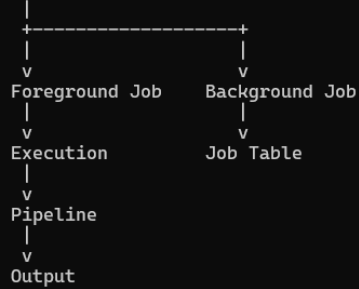
The shell system is divided into several components:

1. Input Module
2. Parser Module
3. Job Management Module
4. Process Creation Module
5. Pipeline Module
6. Signal Handling Module
7. Execution Module

Architecture Diagram

```
User
|
v
Input Module
|
v
Parser
|
v
Command / Arguments
|
v
Job Management
|
```

Process Creation



Execution Path

The user enters a command.

The parser identifies the command and its arguments.

The shell creates a process using `fork()`.

The child process executes the required command using `exec()`.

For pipelines, `pipe()` creates communication channels between processes.

`dup2()` redirects standard input and output to the appropriate pipe.

The parent process uses `waitpid()` to monitor child processes.

Important System Calls

`fork()`

Creates a new child process.

`exec()`

Replaces the child process with the requested program.

`pipe()`

Creates a communication channel between processes.

`dup2()`

Redirects standard input or output.

```

Redirects standard input or output.

### waitpid()

Allows the parent to wait for and monitor a child process.

### kill()

Sends signals to processes when required for job control.

## Pipeline Execution

Example:

seq 1 1000 | grep "[0-9]*$" | wc -l

The output of the first command becomes the input of
the second command.

The output of the second command becomes the input of
the third command.

## Design Decisions

- Pipes are used for inter-process communication.
- fork() is used to create independent processes.
- exec() is used to execute external commands.
- waitpid() is used to monitor process completion.
- Separate process handling makes pipeline execution possible.
- Error conditions are checked after system calls.
- Execution time is measured to analyze performance.

## Error Handling

The program checks for failures from:

- pipe()
- fork()
- dup2()
- exec()
- waitpid()

If a system call fails, an error message is displayed.

## Performance Analysis

The total execution time is measured using
clock_gettime().

Performance can be compared by:

```

```

## Performance Analysis

The total execution time is measured using
clock_gettime().

Performance can be compared by:

- Number of pipeline stages
- Amount of input data
- Number of simultaneous jobs
- Process creation overhead
- Pipe communication overhead

## Conclusion

The system demonstrates how a shell creates processes,
connects them using pipes, executes commands, monitors
their completion, and handles failures.
root@LAPTOP-1C0JKKG6:~# ls -l session23_task2.md
-rw-r--r-- 1 root root 2610 Sep 25 04:00 session23_task2.md
root@LAPTOP-1C0JKKG6:~# =|

```

Observation: I learned how to document a system architecture and execution flow. I understood the role of important Linux system calls in the shell and learned how flow diagrams and design documentation help explain the complete execution process.

Session – 24

Task1: Organize the project source code, improve the build system, create project documentation, verify the repository structure, create a release tag, and validate the final deliverables.

Source code:

```
Ubuntu > root > ossp_2520090049 > skill > Week1 > task1 > C main.c
1  #include <stdio.h>
2
3  int main() {
4      printf("Hello, OSSP Skill Week 1!\n");
5      printf("GCC compilation is working successfully.\n");
6
7      return 0;
8  }
9
```

```
C main.c X M Makefile X
Ubuntu > root > ossp_2520090049 > skill > Week1 > Task1 > M Makefile
1  CC = gcc
2  CFLAGS = -Wall -Wextra
3
4  TARGET = main
5  SOURCE = main.c
6
7  all: $(TARGET)
8
9  $(TARGET): $(SOURCE)
10     $(CC) $(CFLAGS) $(SOURCE) -o $(TARGET)
11
12  run: $(TARGET)
13     ./$$(TARGET)
14
15  clean:
16     rm -f $(TARGET)
17
```

```
C main.c x Makefile x README.md x
Ubuntu > root > ossp_2520090049 > skill > Week1 > Task1 > README.md > # OSSP Skill Week 1 - Task 1
1 # OSSP Skill Week 1 - Task 1
2
3 ## Objective
4
5 To organize source code, improve the build script, add documentation, and verify the project build.
6
7 ## Files
8
9 - main.c - C source program
10 - Makefile - Build script
11 - main - Compiled executable
12
13 ## Compilation
14
15 Build the project using:
16 make
17
18 ## Run
19
20 Run the program using:
21 make run
22
23 ## Clean
24
25 Remove the compiled executable using:
26 make clean
27
28 ## Output
29
30 The program displays:
31 Hello, OSSP Skill Week 1!
32 GCC compilation is working successfully.
33
34 ## Conclusion
35
36 The source code and build process were organized using a Makefile, and the program was successfully compiled and executed.
37
38
```

Output:

```
root@LAPTOP-1COJKKG6:~# cd ~/oss_2520090049/skill/Week1/Task1
code .
root@LAPTOP-1COJKKG6:~/oss_2520090049/skill/Week1/Task1# code main.c
root@LAPTOP-1COJKKG6:~/oss_2520090049/skill/Week1/Task1# code Makefile
root@LAPTOP-1COJKKG6:~/oss_2520090049/skill/Week1/Task1# code README.md
root@LAPTOP-1COJKKG6:~/oss_2520090049/skill/Week1/Task1# make
gcc -Wall -Wextra main.c -o main
root@LAPTOP-1COJKKG6:~/oss_2520090049/skill/Week1/Task1# ./main
Hello, OSSP Skill Week 1!
GCC compilation is working successfully.
root@LAPTOP-1COJKKG6:~/oss_2520090049/skill/Week1/Task1# |
```

Observation: I learned how to organize source code and improve the build process using a Makefile. I understood how to compile, run, and clean a C program using Make commands. I also learned how to create project documentation using a README file, verify the project structure, and use Git commits and tags to manage project versions.

Task2: Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review

Source code:

```

GNU nano 7.2
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void handle_signal(int sig)
{
    if (sig == SIGUSR1)
        printf("SIGUSR1 received successfully.\n");
}

int main()
{
    signal(SIGUSR1, handle_signal);

    printf("OSSP Session 24 - Task 2 Demo\n");
    printf("-----\n");

    printf("Pipeline Demo:\n");
    printf("Command: ls | wc -l\n");

    pid_t pid = fork();

    if (pid == 0)
    {
        execlp("sh", "sh", "-c", "ls | wc -l", NULL);
        return 1;
    }

    waitpid(pid, NULL, 0);

    printf("Pipeline executed successfully.\n");

    printf("\nSignal Demo:\n");
    printf("Sending SIGUSR1 to the current process...\n");

    kill(getpid(), SIGUSR1);

```

```

    printf("Session 24 Task 2 demonstration completed.\n");

    return 0;
}

```

Output:

```

root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# gcc -Wall -Wextra session24_task2.c -o session24_task2
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# ls -la
total 28
drwxr-xr-x 2 root root 4096 Sep 25 05:56 .
drwxr-xr-x 4 root root 4096 Sep 25 05:13 ..
-rwxr-xr-x 1 root root 16264 Sep 25 05:56 session24_task2
-rw-r--r-- 1 root root 812 Sep 25 05:55 session24_task2.c
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# ./session24_task2
OSSP Session 24 - Task 2 Demo

-----
Pipeline Demo:
Command: ls | wc -l
2
Pipeline executed successfully.

Signal Demo:
Sending SIGUSR1 to the current process...
SIGUSR1 received successfully.
Session 24 Task 2 demonstration completed.
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# nano README.md
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# ls -la
total 32
drwxr-xr-x 2 root root 4096 Sep 25 05:58 .
drwxr-xr-x 4 root root 4096 Sep 25 05:13 ..
-rw-r--r-- 1 root root 1234 Sep 25 05:58 README.md
-rwxr-xr-x 1 root root 16264 Sep 25 05:56 session24_task2
-rw-r--r-- 1 root root 812 Sep 25 05:55 session24_task2.c
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# git status --short
?? ../Task1/main
?? ./
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# ls -la
total 32
drwxr-xr-x 2 root root 4096 Sep 25 05:58 .
drwxr-xr-x 4 root root 4096 Sep 25 05:13 ..
-rw-r--r-- 1 root root 1234 Sep 25 05:58 README.md
-rwxr-xr-x 1 root root 16264 Sep 25 05:56 session24_task2
-rw-r--r-- 1 root root 812 Sep 25 05:55 session24_task2.c
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# rm ../Task1/main
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# git status --short
?? ./
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# git add .
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# git status
On branch ossp_skill
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   README.md
    new file:   session24_task2
    new file:   session24_task2.c

root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# git commit -m "Complete Session 24 Task 2"
[ossp_skill 04f73f2] Complete Session 24 Task 2
Committer: root <root@LAPTOP-1C0JKKG6.localdomain>
Your name and email address were configured automatically based
on your username and hostname. Please check that they are accurate.
You can suppress this message by setting them explicitly. Run the
following command and follow the instructions in your editor to edit
your configuration file:

    git config --global --edit

After doing this, you may fix the identity used for this commit with:

    git commit --amend --reset-author

3 files changed, 89 insertions(+)
create mode 100644 skill/Week1/Task2/README.md
create mode 100755 skill/Week1/Task2/session24_task2
create mode 100644 skill/Week1/Task2/session24_task2.c
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# git status
On branch ossp_skill
nothing to commit, working tree clean
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# git log --oneline --decorate -3
04f73f2 (HEAD -> ossp_skill) Complete Session 24 Task 2
a2a0989 (tag: v1.0) Complete Session 24 Task 1
dde62fe (origin/main, origin/HEAD, main) Add files via upload
root@LAPTOP-1C0JKKG6:~/ossp_2520090049/skill/Week1/Task2# |

```

Observation: I learned how to demonstrate Linux pipelines and signal handling in a project review. I understood how commands can be connected using pipelines and how signals such as SIGUSR1 can be sent and handled by a process. I also learned how to organize, validate, document, and commit project deliverables using Git.